Assignment 4: Heap Data Structures Implementation, Analysis, and Applications

Hasaan Ali

Overview

This assignment implements Heapsort using an array-based max heap and builds a max-heap priority queue for task scheduling. It also analyzes the time and space complexity of the algorithms and compares Heapsort empirically with Quicksort and Merge Sort on sorted, reverse-sorted, and random data.

1. Heapsort Implementation

Heapsort is a comparison-based sorting algorithm that uses the binary heap data structure. The implementation first rearranges the input into a max heap, where every parent is greater than or equal to its children. The maximum value is then located at index 0. The algorithm repeatedly swaps that value with the last unsorted element, shrinks the active heap, and restores the max-heap property with heapify.

1.1 Array Representation of the Heap

An array is a natural representation for a complete binary tree because no explicit node pointers are needed. For a node at index i, the left child is at 2i + 1, the right child is at 2i + 2, and the parent is at (i - 1) // 2. This gives constant-time access to relatives in the heap and keeps the implementation compact.

1.2 Main Steps

- Build a max heap by calling heapify on each non-leaf node from the bottom upward.
- Swap the root, which contains the maximum element, with the final element in the unsorted region.
- Reduce the heap size by one so the maximum element remains fixed in its final position.
- Call heapify on the root to restore the max-heap property.
- Repeat until only one element remains in the heap.

2. Heapsort Complexity Analysis

2.1 Time Complexity

Building the initial max heap takes O(n) time. Although a single heapify operation can take O(log n), most nodes are close to the leaves and require very little work. After the heap is built, the algorithm performs n - 1 extractions. Each extraction swaps the root with the end of the active heap and performs heapify, which costs O(log n). Therefore the sorting phase costs O(n log n), and the total running time is O(n log n).

Case	Time Complexity	Reason
Best	O(n log n)	Even favorable input still requires the same sequence of heap extractions and heap restoration.
Average	O(n log n)	Each extraction may move an element through a heap whose height is logarithmic.
Worst	O(n log n)	Heapify is bounded by the heap height, and n extractions are performed.

Unlike some forms of Quicksort, Heapsort does not become quadratic when the input is already sorted or arranged adversarially. The heap structure guarantees logarithmic-height restoration work after each extraction, so O(n log n) is maintained across best, average, and worst cases.

2.2 Space Complexity and Overhead

The sorting itself is in place: elements are rearranged inside the original array, so no second array proportional to n is required. The iterative storage overhead is O(1). In this implementation, heapify is recursive, so recursion can use O(log n) call-stack space in the worst case. An iterative heapify implementation could reduce that call-stack overhead to O(1). Heapsort also performs many swaps and non-sequential accesses between parents and children, which can make it less cache-friendly than some alternatives.

3. Empirical Comparison with Quicksort and Merge Sort

The benchmark tests input sizes of 1,000, 5,000, and 10,000 elements. For every size, the same underlying values are arranged as sorted, reverse-sorted, and random inputs. Each algorithm is run three times per case and the average wall-clock time is recorded with time.perf_counter(). The purpose is not to declare a universal fastest algorithm, but to compare practical behavior with the theoretical analysis.

Distribution	Heapsort (s)	Quicksort (s)	Merge Sort (s)
sorted	0.027937	0.006138	0.010863
reverse-sorted	0.025960	0.006339	0.010952
random	0.026689	0.012527	0.017319

For n = 10,000, Heapsort took about 0.026-0.028 seconds across all three distributions. Its time remained relatively stable as the ordering changed, which is consistent with its O(n log n) behavior in all cases. The Quicksort implementation was the fastest in these tests, especially on sorted and reverse-sorted input because the middle-element pivot kept partitions balanced. Merge Sort was also faster than Heapsort in this Python implementation but required additional arrays during merging.

Random input increased the running time of Quicksort and Merge Sort more noticeably than it increased Heapsort. This does not change their asymptotic classifications; it reflects implementation details, data movement, branch behavior, Python overhead, and cache locality. The measured times should therefore be interpreted as one empirical sample rather than absolute performance guarantees.

4. Priority Queue Design

4.1 Data Structure Choice

The priority queue is implemented as a max heap stored in a Python list. A max heap was selected because the scheduler is designed to run the task with the highest numeric priority first. The list provides O(1) access by index and maps directly to the binary-heap formulas for parent and child positions. A separate dictionary maps each task ID to its current heap index, allowing a task to be found quickly when its priority changes.

4.2 Task Representation

Each Task stores a task ID, priority, arrival time, and optional deadline. The priority determines heap ordering. Arrival time and deadline are included because they are common scheduling attributes and could be used to build more advanced policies later.

4.3 Core Operations and Complexity

Operation	Complexity	Explanation
insert(task)	O(log n)	Append at the end, then sift upward through at most the height of the heap.
extract_max()	O(log n)	Move the last task to the root, then sift downward through at most the heap height.
increase_key()	O(log n)	Update the priority and sift upward if the task became more important.
decrease_key()	O(log n)	Update the priority and sift downward if the task became less important.
is_empty()	O(1)	Check whether the heap list contains zero elements.

The heap height is floor(log2 n), so insertion, extraction, and priority changes are logarithmic. The positions dictionary adds O(n) memory overall but makes locating a task by ID O(1) on average before the heap adjustment occurs. Without this dictionary, finding an arbitrary task for a key update would require O(n) search time.

5. Scheduler Simulation

The scheduler demonstration inserts five tasks with different priorities. Before execution, task T1 has its priority increased from 3 to 9 to model a task becoming more urgent. The queue then repeatedly calls extract_max(). Because this is a max-heap, tasks leave the queue from highest numeric priority to lowest numeric priority.

Task	Final Priority	Arrival Time	Deadline
T4	10	3	5
T1	9	0	10
T2	8	1	6
T3	5	2	12
T5	4	4	15

This simulation illustrates why heaps are useful for scheduling: the scheduler does not need to fully sort the entire task list after every insertion. It only maintains enough structure to access and remove the highest-priority task efficiently.

6. Real-World Applications

- **Operating-system scheduling:** A priority queue can select the most important runnable process or job while allowing priorities to change over time.
- **Event-driven simulation:** Future events can be stored by priority or execution time so that the next event is obtained efficiently.
- **Network and service queues:** Requests can be prioritized by urgency, service class, or deadline.
- **Graph algorithms:** Algorithms such as Dijkstra's shortest path and Prim's minimum spanning tree rely on priority-queue operations.
- **Top-k and streaming problems:** A heap can maintain only the most important k values without sorting the complete data set repeatedly.

7. Conclusion

Heaps provide a compact way to maintain partial order using an array. Heapsort uses this property to guarantee O(n log n) time regardless of the initial input distribution and can sort in place. The same structure also supports an efficient priority queue in which insertion, removal of the highest-priority task, and key changes are O(log n). The empirical results were consistent with the theoretical analysis: Heapsort showed stable performance across input distributions, although the particular Python implementation was slower than the tested Quicksort and Merge Sort implementations because asymptotic complexity does not capture constant factors and memory-access behavior.

Appendix: Complete Benchmark Results

n	Distribution	Algorithm	Average Time (s)
1000	sorted	Heapsort	0.002094
1000	sorted	Quicksort	0.000444
1000	sorted	Merge Sort	0.000887
1000	reverse-sorted	Heapsort	0.001735
1000	reverse-sorted	Quicksort	0.000479
1000	reverse-sorted	Merge Sort	0.000892
1000	random	Heapsort	0.001749
1000	random	Quicksort	0.001006
1000	random	Merge Sort	0.001238
5000	sorted	Heapsort	0.013334
5000	sorted	Quicksort	0.003063
5000	sorted	Merge Sort	0.005103
5000	reverse-sorted	Heapsort	0.012773
5000	reverse-sorted	Quicksort	0.003443
5000	reverse-sorted	Merge Sort	0.005462
5000	random	Heapsort	0.012296
5000	random	Quicksort	0.006668
5000	random	Merge Sort	0.008582
10000	sorted	Heapsort	0.027937
10000	sorted	Quicksort	0.006138
10000	sorted	Merge Sort	0.010863
10000	reverse-sorted	Heapsort	0.025960
10000	reverse-sorted	Quicksort	0.006339
10000	reverse-sorted	Merge Sort	0.010952
10000	random	Heapsort	0.026689
10000	random	Quicksort	0.012527
10000	random	Merge Sort	0.017319